Phase 1 | Master Project Overview

Understand the product, its architecture, and how to explain it.

React/Vite Edition | Repository-grounded study material | Snapshot: 2026-10-24

1. Project in plain English

Ready Scout is a disaster-preparedness learning platform. Its React web app brings together preparedness content, learner progress, an earthquake simulation, alerts, an emergency-services locator, and a disaster-management chatbot. The main API is built with Express and stores user records in MongoDB Atlas. A separate Flask service connects the chatbot to Gemini.

A concise framing: Ready Scout helps learners explore disaster-preparedness material and interactive practice in one web application, supported by alerts and nearby resource discovery.

2. Problem and intended users

The project addresses fragmented access to educational preparedness resources. Students can review learning modules, practice earthquake responses, track progress, and use reference tools. The repository also contains a teacher route and role field, but the platform should be described as a learning product rather than a verified emergency-response system.

3. Main workflows

- The browser loads the React application through Vite in development or static hosting in production. React Router maps URLs to home, registration, login, dashboards, learning modules, chatbot, nearby services and CAP alert pages.
- Registration and login call Express endpoints. The backend validates fields, hashes passwords with bcryptjs, writes user records via Mongoose, and returns a signed JWT. Passport handles Google OAuth when credentials are configured.
- The alert ticker requests normalized NDMA SACHET summaries from Express; CAP details are fetched and parsed from XML. The repository also contains client utilities for USGS earthquake and GDACS flood feeds.
- The nearby page requests browser location, uses a New Delhi fallback if location is unavailable, fetches nearby place records, sorts by distance and displays markers using Leaflet.
- The earthquake drill is a Phaser 3 browser game. A separate earthquake-quiz project is implemented as its own React/Vite app.
- The chatbot UI sends messages to Flask, which adds disaster-education context and calls Gemini.

4. Architecture map

Layer	Current implementation
Web client	React 19, Vite, React Router, Tailwind CSS
Application API	Node.js and Express 5
Persistence	MongoDB Atlas through Mongoose
Authentication	JWT, bcryptjs, Passport.js and Google OAuth 2.0
AI service	Flask/Python calling Gemini generateContent
Map and learning	Leaflet, OpenStreetMap tiles, Geoapify Places, Phaser 3
Hosting configuration	Vercel frontend and Render backend; Express can also serve frontend/dist

5. Data and request flow

The browser owns presentation and user interactions. Express owns account operations and selected integrations, then translates provider responses into frontend-friendly shapes. MongoDB Atlas persists user records. Flask owns the Gemini call. Leaflet renders tiles and markers; it is separate from the Geoapify Places provider that supplies nearby points.

6. Scope and limitations

- This is an educational prototype, not a dispatch system or official source for emergency decisions.
- Nearby search depends on a valid backend GEOAPIFY_API_KEY and provider availability. Do not describe the locator as live or operational until the deployed key and endpoint have been verified.
- Place coverage depends on OpenStreetMap-derived data and category tagging; absence of a result does not prove a facility is absent.
- The repository contains fire and flood learning pages, but describe them as scaffolded if their full interactive learning workflows are not implemented.
- The auth code stores a token in browser localStorage and sends OAuth token material through a redirect URL; these are design areas to improve.

One-minute answer: “Ready Scout is a React/Vite disaster-preparedness learning platform with an Express API and MongoDB Atlas. It combines learning modules, an earthquake simulation, alert integrations, a nearby emergency-services map, and a Flask/Gemini chatbot. The integrations depend on external feeds and credentials, so I distinguish educational features from guaranteed emergency coverage.”